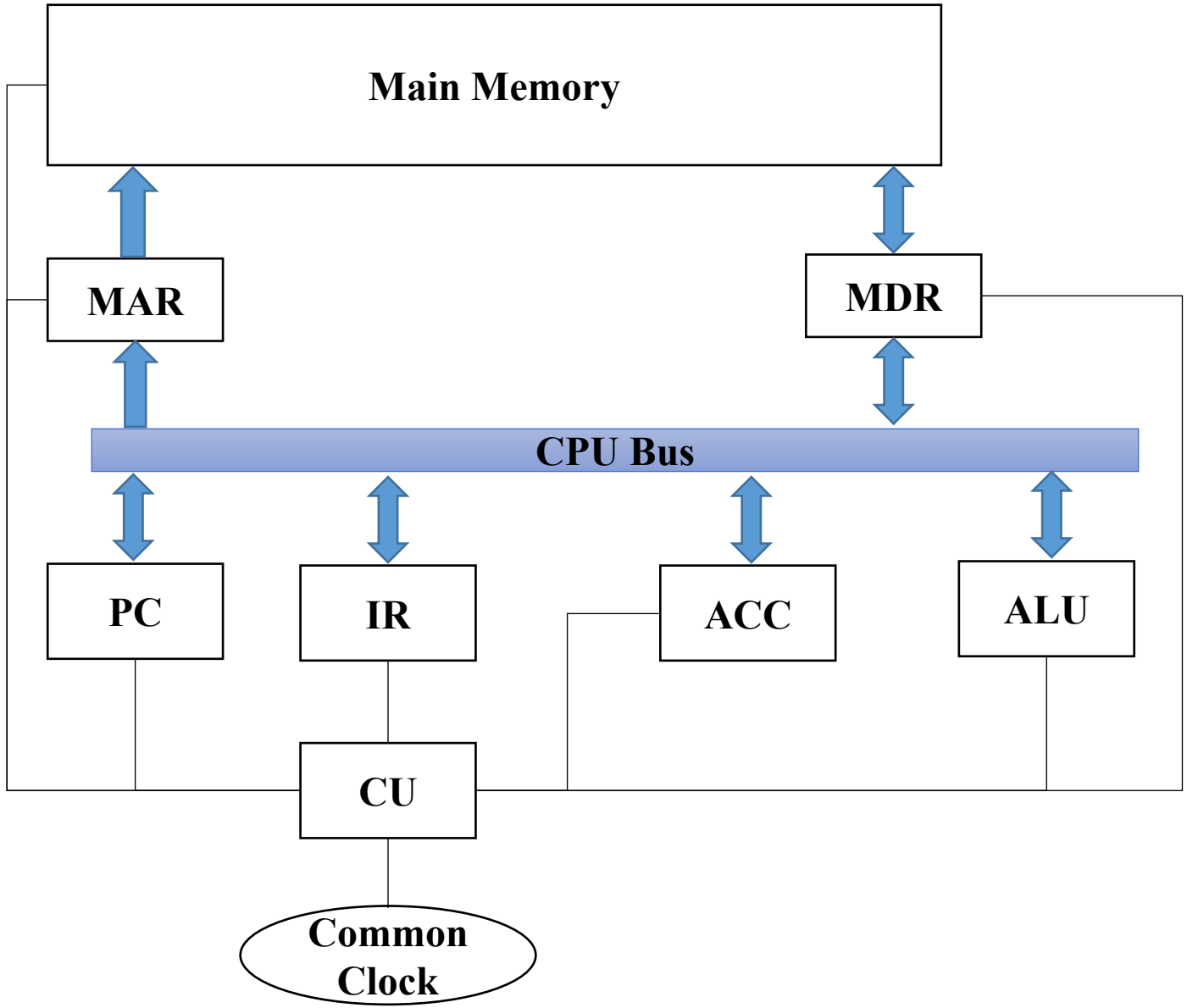
Pipelining : Basic Concepts

CSE Sem-4

CPU Configuration and Instruction Execution Operations



1. Address of the next instruction is transferred from PC to MAR. The instruction is located in MM.
2. Instruction is copied from memory to MDR
3. Instruction is transferred to IR for decoding.
4. Control unit send signals to appropriate devices (ALU, ACC, memory) to execute the instruction.
5. The result is stored.

Why Pipelining?

- CPU performance can be improved by:
  - Improve the hardware by introducing faster circuits.
  - Arrange the hardware such that more than one operation can be performed at the same time.

- **Pipelining or Pipeline Processing:**
  - Pipelining is a technique where **multiple instructions are overlapped during execution**.
  - Pipelining is a process of **arrangement of hardware elements of the CPU** such that its overall performance is increased.
  - It allows **storing and execution of instructions in an orderly process**.
  - Pipeline **has two ends**, the **input end and the output end**. Between these ends, there are multiple stages/segments such that output of one stage is connected to input of next stage and each stage performs a specific operation.
  - **Stages are purely a combination of sequential and combinational circuits** performing arithmetic and logic operations over the data stream flowing through the pipe.

Pipeline vs non-pipeline architecture:

p1	F	D	O	E	S										
p2	-	-	-	-	-	F	D	O	E	S					
p3	-	-	-	-	-	-	-	-	-	-	F	D	O	E	S
Clk	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

F = Instruction Fetch
D = Decode
O = Operand Fetch
E = Execute
S = Storing result

Fig: 1 Non-pipelined architecture

Space or stages

S1	F1	F2	F3				
S2		D1	D2	D3			
S3			O1	O2	O3		
S4				E1	E2	E3	
S5					S1	S2	S3
Clk	1	2	3	4	5	6	7

Time

Fig: 2 Pipelined architecture
(Space time diagram)

If there are 'K' stages in the pipelined architecture, and number of instructions are 'n' – it will take K clock pulses to perform the 1st instruction, and after, at each clock pulse 1 instructions will be completed.

So number of clock pulse required to perform n instructions in K stage pipelined architecture:

$$= K+(n-1)$$

Speed up of pipelined architecture=

S = CP in non-pipeline / CP in pipeline

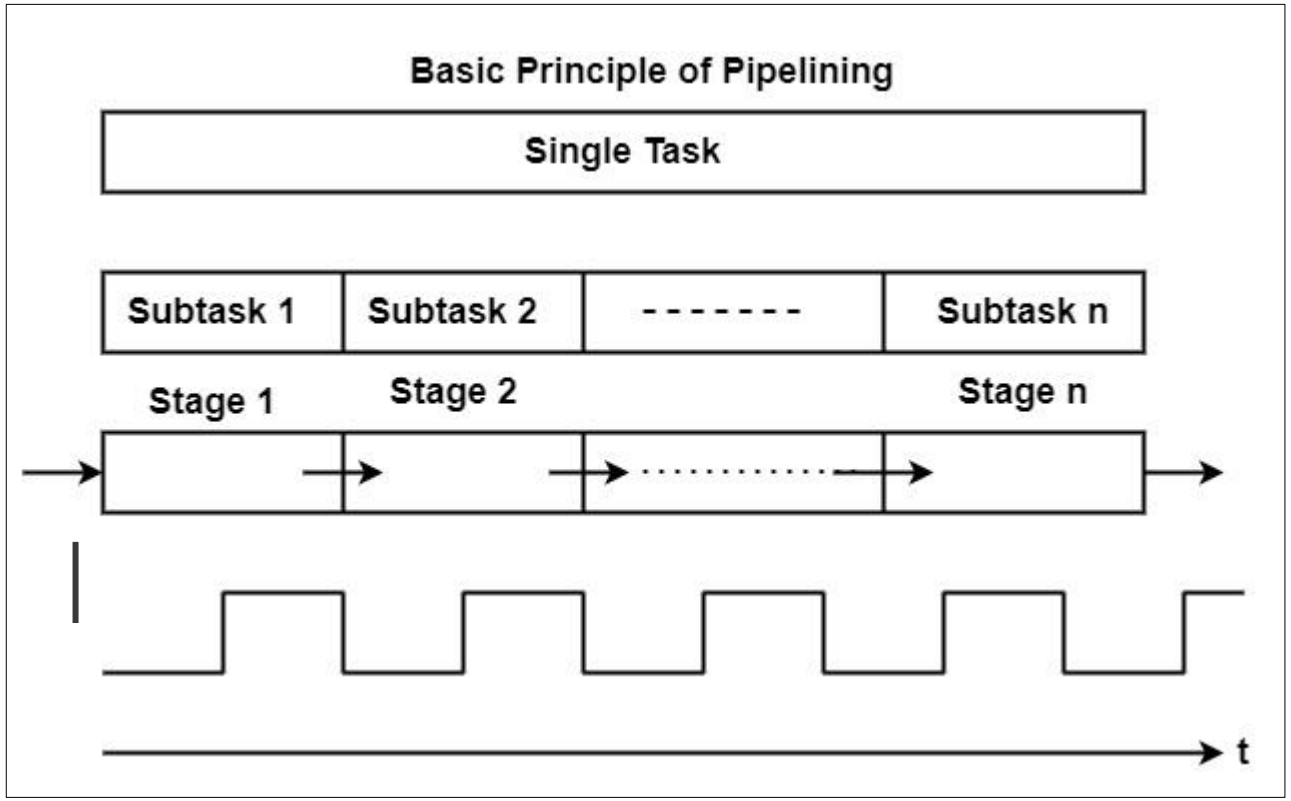
Difference Between Pipeline and Non-pipeline Architecture

Pipeline Architecture	Non-pipeline Architecture
Pipelining is an implementation technique where multiple instructions are overlapped in execution.	All the actions (fetching, decoding, execution of instructions and storing results into memory) are grouped into a single step.
Many instructions are executed at the same time.	Only one instruction is executed at a time.
Execution time is comparatively less, and execution is done in a fewer cycles.	Execution takes more time or more number of cycles comparatively.
It has a high throughput (number of instructions executed per unit time).	It has a low throughput.
The pipeline is filled by the CPU scheduler from a pool of work which is waiting to occur.	In a Non-Pipelining system, the CPU scheduler chooses the instruction from the pool of waiting instructions, when an execution unit gives a signal that it is free.

CPU Scheduling is **a process of determining which process will own CPU for execution while another process is on hold**.

Principle of Pipeline

- The **problem is divided into a series of tasks** that have to be completed one after another.
- Each **subtask can be executed by a hardware** that **operates concurrently** with other pipeline stages.
- All **pipeline stages works sequentially**, receiving their input from the previous stage and transferring their output to the next stage.



- There is a **constant stream of tasks** into the pipe and there is overlapped execution at the subtask level.
- Each **stage gets a new input at the beginning of the clock cycle**, each stage has **a single clock cycle available for implementing the needed operations**, and each **stage produces the result to the next stage** by the starting of the subsequent clock cycle.

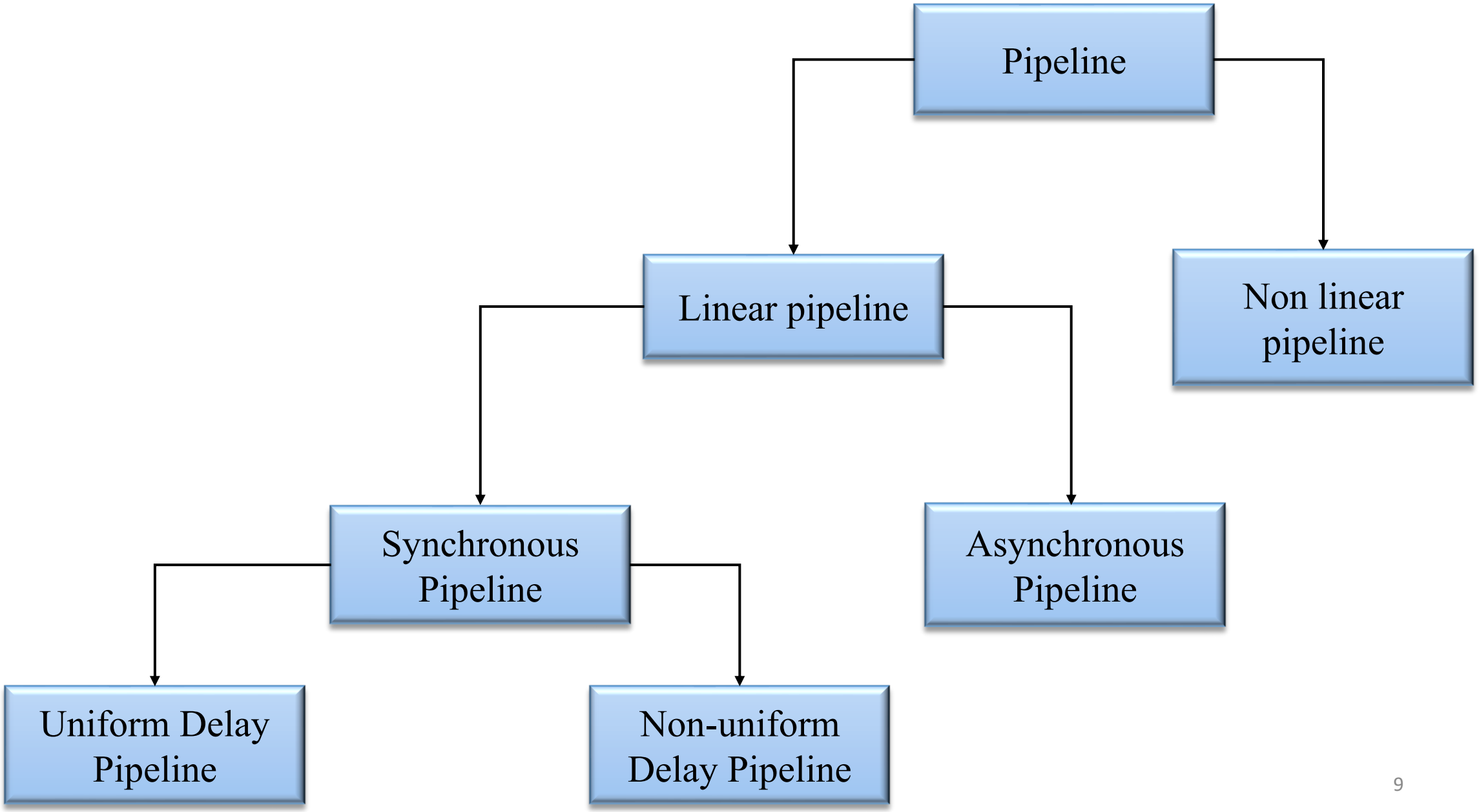
Advantages of Pipeline

- It can **reduce the number of cycles** to perform multiple instructions.
- It can raise the multiple instructions that can be processed together and **lower the delay** between completed instructions, which can **increase the throughput** of the system.
- The more pipeline stages a processor has, the more instructions it can process **at once**. Today's microprocessor manufacturer uses 2 to 40 stages pipeline.
- If pipelining is used, the ALU can **perform faster**, but design will be more complex.
- Pipelining in theory **increases performance** over an un-pipelined core by a factor of the number of stages.
- Pipelined CPUs generally work at a higher clock frequency than the RAM clock frequency, increasing computers overall performance.

Disadvantages of Pipeline

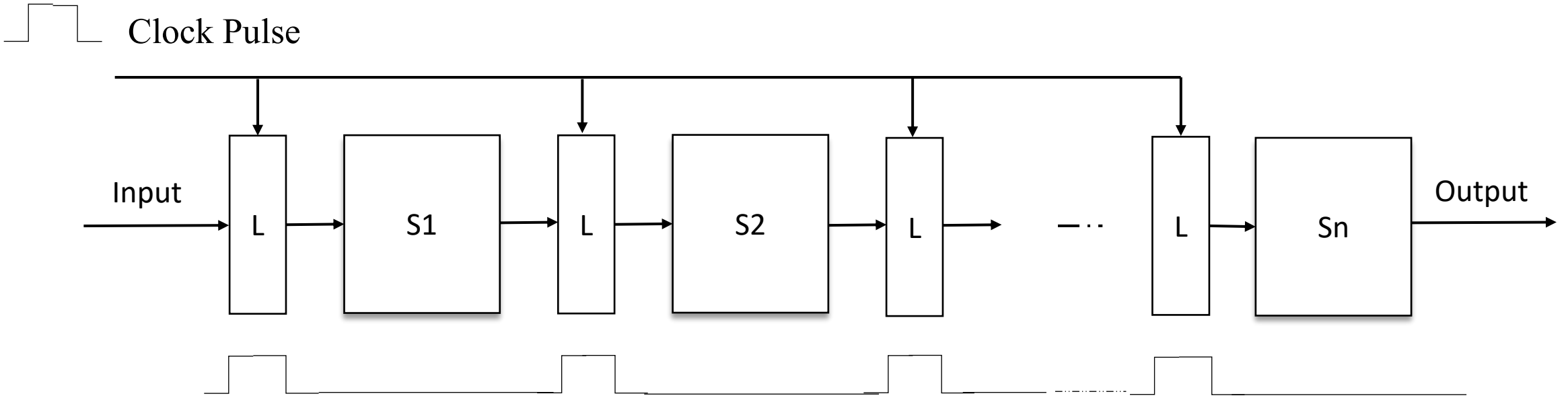
- The **design of pipelined processor is complex and costly**, compared to non-pipelined processor.
- A non-pipelined processor will have a defined instruction throughput. The **performance of a pipelined processor is much harder to predict** and may vary widely for different programs.
- In pipelining **every branching operation is delayed**, the entire pipeline must be cleared, as the processor cannot know in advance from where to read the next instruction and must wait for the branch instruction to finish, leaving the pipeline empty.
- Problems occur when serial instructions being executed concurrently. Any instruction in the pipeline might depend on the output of the previous instruction, causing the pipeline control logic to wait and insert a wasted clock cycle into the pipeline until the dependency is resolved.

Types of Pipeline



Synchronous Pipeline Model

- Each stage consists of an input buffer/latch, followed by a combinational circuit.
- The **latches** are faster registers for holding the intermediate results between the stages.
- The output of combinational circuit is applied to the input latch of the next segment.
- Each latch is synchronized with same clock pulse. When i/p is high, latch will transfer data to next stage simultaneously.



Difference between latch and register: A latch loses the information (data) when passed on to the next stage. A register retains the information until it cleared.

Types of Synchronous Pipeline

❖ **Uniform delay pipeline:** In this type of pipeline, all the stages will take same time to complete an operation.

In uniform delay pipeline,

Cycle Time (Tp) = Latch Delay + Stage Delay

❖ **Non-Uniform delay pipeline:** In this type of pipeline, different stages take different time to complete an operation.

In this type of pipeline,

Cycle Time (Tp) = Latch Delay + Maximum (Stage Delay)

For example, if there are 4 stages with delays, 1 ns, 2 ns, 3 ns, and 4 ns, then

Tp = Maximum (1 ns, 2 ns, 3 ns, 4 ns) + Latch Delay

= 4 ns + Latch Delay

Performance Metrics for Pipeline

❖ **Clock Period:** It is the **time required to complete a single stage (latch delay + stage delay).** Time delay for each interface latch is similar. But time delay for different combinational circuits or different stages is different. So, maximum of all stage duration should be taken as common stage duration.

Total clock period for pipeline architecture:

$$\tau = \max \{\tau i\}_{i=1}^{k} + \tau l \quad \text{for i = 1 to k}$$

where τi is the time delay for stage i

τl is the time delay for latch

❖ **Speed-up:** Speed-up of k-stage pipeline processor over an equivalent pipelined processor as:

$S_k = T_1/T_k = nk / k+(n-1)$ where n is the number of instructions,
k is the number of stages in each instruction.

Performance Metrics for Pipeline

❖ **Efficiency:** Efficiency of a linear pipeline is measured by **the ratio of the busy time-space spans over the total time-space span.**

Total time-space span = number of clock pulses for pipeline * total stages

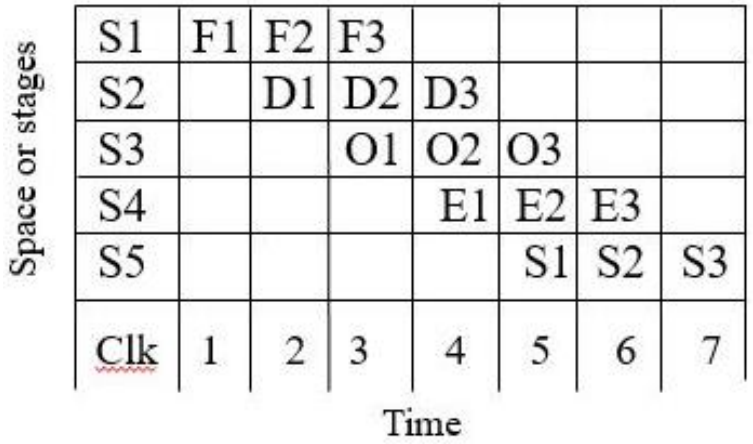
= [k+(n-1)] * k

Busy time-space span = number of instructions * number of stages

= n*k

Efficiency η = [n*k] / [k+(n-1) * k] = n / k+(n-1)

= number of instructions / number of clock pulses for pipeline



- Larger the number of tasks flowing through the pipeline, will increase the efficiency.

❖ **Throughput:** The number of instructions that can be completed by a pipeline per unit time.

W = number of instructions / [number of clock pulses for pipeline * clock period]

= n / [k+(n-1)] * τ

= η / τ

Numerical on Synchronous Pipeline

Question 1: Consider a 4-segment pipeline with stage delays (2 ns, 8 ns, 3 ns, 10 ns). Find the time taken to execute 100 tasks in the above pipeline. [Consider that there is no latch delay]

Answer: CPU time for Pipeline= (k + n – 1) Tp [k = stages, n = tasks, Tp = Clock cycle time]

As the above pipeline is a non-uniform pipeline, Tp = max(2, 8, 3, 10) = 10 ns

CPU time for Pipeline = (4 + 100 – 1) 10 ns = 1030 ns

Question 2: A 4-stage pipeline with stage delays 150 ns, 120 ns, 160 ns, 140 ns. Latches have a delay of 5 ns each. Find the time taken to execute 1000 tasks in the pipeline.

Answer: CPU time for Pipeline= (k + n – 1) Tp [k = stages, n = tasks, Tp = Clock cycle time]

For non-uniform pipeline, Tp = max(150, 120, 160, 140) = 160 ns

CPU time for Pipeline = (4 + 1000 – 1) (160+5) ns

= 1003 x 165 ns = 165,495 ns = 165.5 μs

Asynchronous Pipeline Model

- **No latches** are there in this mode.
- The data flow between adjacent stages is controlled by a **handshaking protocol**.
- When the stage S_i is ready to transmit intermediate data, it sends a **ready signal** to next stage S_{i+1}
- Stage S_{i+1} sends an **acknowledgement signal** that it's ready to accept the incoming data
- Stage S_i will send the data to S_{i+1}

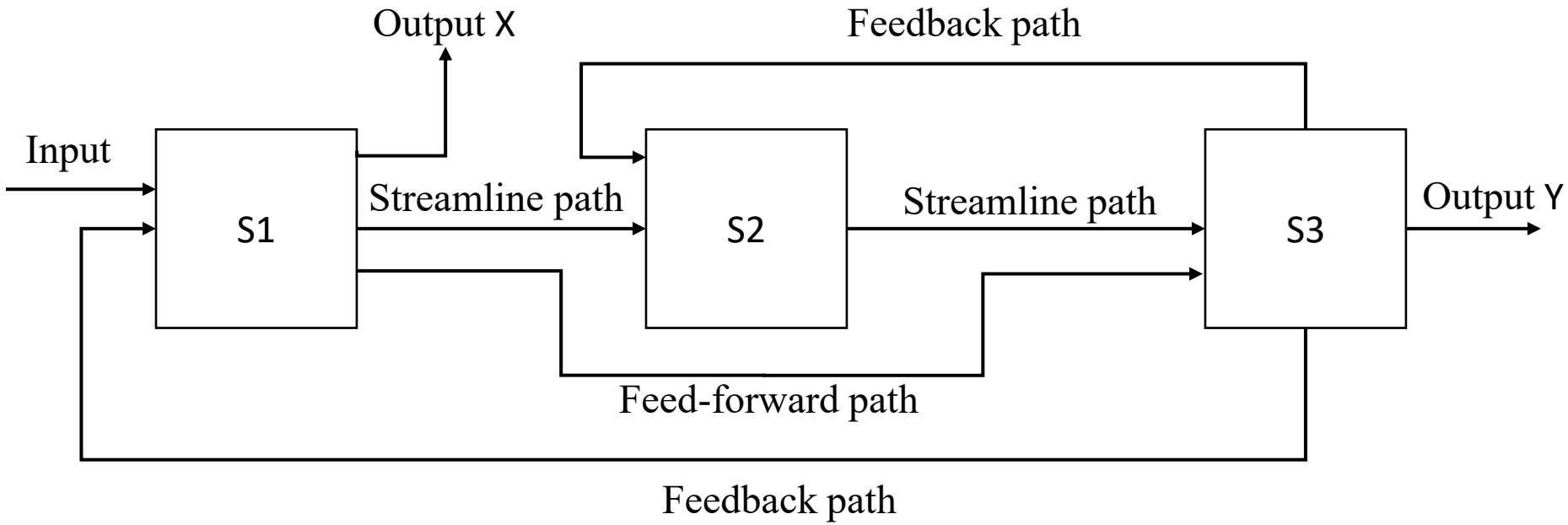


Non-linear Pipeline Model

- A pipeline containing **feed-forward** and **feedback connections** in addition to the **streamline connection** is called a non-linear pipeline.

- Due to the feed-forward and feedback connections, it can be reconfigured to perform different functions at different time. Due to this, it also called **Dynamic Pipeline**. In this pipeline, it is not necessary that the output will be coming from last stage only. At any stage output can be generated for different functions.

- Non-linear pipeline are used for recursion problems.



Difference Between Linear and Non-linear Pipeline

Linear pipeline	Non linear pipeline
Linear pipeline are **static pipeline** because they are used to perform fixed functions.	Non-Linear pipeline are **dynamic pipeline** because they can be reconfigured to perform variable functions at different times.
Linear pipeline allows **only streamline** connections.	Non-Linear pipeline allows **feed-forward and feedback connections** in addition to the **streamline connection.**
It is relatively **easy** to partition a given function into a sequence of linearly ordered sub functions.	Function partitioning is relatively **difficult** because the pipeline stages are interconnected with loops in addition to streamline connections.
The **output** of the pipeline is produced **from the last stage.**	The **output** of the pipeline **is not necessarily produced from the last stage.**

Li and Ramamurthy's Classification

According to the configuration of a pipeline and control strategies, Li and Ramamurthy (1977) classify pipelines under three schemes:

- ❑ Unifunction vs Multifunction pipeline
- ❑ Static vs Dynamic pipeline
- ❑ Scalar vs Vector pipeline

➢ ***Unifunction vs Multifunction pipeline:***

❖ **Unifunction pipeline:** A pipeline unit with **fixed and dedicated function** is called as a unifunction pipeline.

Example: Adder unit, logical unit, shift unit.

❖ **Multifunction pipeline:** A pipeline that performs **different functions** either at different times or at the same time, by **interconnecting different subset of stages** in the pipeline, is called as a multifunction Pipeline.

Example: IT-ASC (Supercomputer) processor has 4 multifunctional pipeline processors, each of which can be reconfigured to perform multiple arithmetic or logical operations. Any mixture of arithmetic and logical operations can be executed simultaneously.

Static vs Dynamic pipeline

❖ **Static pipeline:**

- It is a pipeline that assumes only one functional configuration at a time.
- Static pipelines are preferred when instructions of same type are to be executed continuously.
- The functions performed by static pipeline should not be changed frequently. The frequent change in the pipeline will lead to degradation of performance of static pipeline.
- A unifunction pipeline must be static.

❖ **Dynamic pipeline:**

- Several functional configurations can exist simultaneously.
- The control in the dynamic pipeline is complex as compared to the control in the static pipeline.
- A multi-function pipeline must be dynamic.

Scalar vs Vector pipeline

❖ **Scalar pipeline:**

- It processes a sequence of scalar operands under the control of a loop.
- Instructions in a small loop are often prefetched into the instruction buffer.
- The scalar operands which are required for repeated scalar instructions are moved into a data cache, to continuously supply the pipeline with operands.

❖ **Vector pipeline:**

- They are specially designed to handle vector instructions. A vector operand contains an ordered set of n elements, where n is called the length of the vector. All elements in a vector are same type scalar quantities, which may be a floating point number, an integer, a logical value or a character.
- Vector pipelines are expanded version of the scalar pipelines.